

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

WEEK 3 LECTURE 2

MEMORY
ADDRESSING

WISH YOU ALL THE BEST

SOC 3010

OPERATING SYSTEM

WEEK 3 LECTURE 2

MEMORY

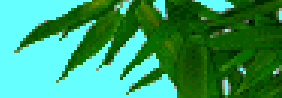
ADDRESSING

SEGMENTATION UNIT

Memory Addressing

- ★ **Memory Addresses**
 - Programmers casually refer to a memory address as the way to access the contents of a memory cell.
- ★ **But when dealing with Intel 80x86 microprocessors, we have to distinguish among three kinds of addresses:**
 - **Logical address**
 - **Linear address**
 - **Physical address**

Memory Addressing



* Logical address

- Included in the machine language instructions to specify the address of an operand or of an instruction.
- This type of address embodies the well-known Intel segmented architecture that forces MS-DOS and Windows programmers to divide their programs into segments.
- **Each logical address consists of a segment and an offset (or displacement) that denotes the distance from the start of the segment to the actual address.**

* Linear address

- **A single 32-bit unsigned integer that can be used to address up to 4 GB, that is, up to 4,294,967,296 memory cells.**
- Linear addresses are usually represented in hexadecimal notation; their values range from **0x00000000 to 0xFFFFFFFF**.

* Physical address

- **Used to address memory cells included in memory chips.**
- They correspond to the electrical signals sent along the address pins of the microprocessor to the memory bus.
- **Physical addresses are represented as 32-bit or 36-bits unsigned integers.**



Memory Addressing

□ Logical address translation

- ★ The **Memory Management Unit(MMU)** transforms a **logical address** into a **linear address** by means of a hardware circuit called a **segmentation unit**
- ★ Subsequently, a second hardware circuit called a **paging unit** transforms the **linear address** into a **physical address**



Memory Addressing

□ MEMORY ARBITER

- ★ In multiprocessor systems, all CPUs usually share the same memory; this means that RAM chips may be accessed concurrently by independent CPUs.
- ★ Because read or write operations on a RAM chip must be performed serially, a hardware circuit called a memory arbiter is inserted between the bus and every RAM chip.
- ★ Its role is to grant access to a CPU if the chip is free and to delay it if the chip is busy servicing a request by another processor.
- ★ Even uniprocessor systems use memory arbiters , because they include specialized processors called DMA controllers that operate concurrently with the CPU
- ★ In the case of multiprocessor systems, the structure of the arbiter is more complex because it has more input ports.
- ★ The dual Pentium, for instance, maintains a two-port arbiter at each chip entrance and requires that the two CPUs exchange synchronization messages before attempting to use the common bus.

Memory Addressing

❑ Segmentation in Hardware

- ★ Starting with the 80386 model, Intel microprocessors perform address translation in two different ways called **real mode** and **protected mode**.
- ★ Real mode exists mostly to maintain processor compatibility with older models and to allow the operating system to bootstrap.

❑ Segment Selectors and Segmentation Registers

- ★ A logical address consists of two parts: a **segment identifier** and an **offset** that specifies the relative address within the segment.
- ★ The segment identifier is a 16-bit field called **Segment Selector**, while the offset is a 32-bit field.
- ★ To make it easy to retrieve segment selectors quickly, the processor provides **segmentation registers** whose only purpose is to hold Segment Selectors; these registers are called **cs, ss, ds, es, fs, and gs**

Memory Addressing



- * Although there are only six of the SEGMENT registers, a program can reuse the same segmentation register for different purposes by saving its content in memory and then restoring it later.
- * Three of the six segmentation registers have specific purposes:
 - ❖ **CS** The code segment register, which points to a segment containing program instructions
 - ❖ **SS** The stack segment register, which points to a segment containing the current program stack
 - ❖ **DS** The data segment register, which points to a segment containing global and static data
- * The remaining three segmentation registers are general purpose and may refer to arbitrary data segments.
- * The **CS** register has another important function: it includes a 2-bit field that specifies the Current Privilege Level (CPL) of the CPU.
- * The value 0 denotes the highest privilege level, while the value 3 denotes the lowest one.
- * Linux uses only levels 0 and 3, which are respectively called Kernel Mode and User Mode.



Memory Addressing



- ❑ **SEGMENT SELECTORS & OFFSET**
- ★ Logical Address generated by the processor consists of **TWO PARTS**: Segment Selector or Identifier and Offset
- ★ Segment Selector or address is stored in one of the Segment Registers – CS, DS, SS, ES, FS, or GS and
- ★ Offset is stored in one of the following registers – IP, SP, SI or DI
- ★ In order to access any SEGMENT, the logical address generated by the CPU will be available in the following Segment Selector & Offset

SEGMENT	LOGICAL ADDRESS	
	SEGMENT SELECTOR REGISTER(16bits)	OFFSET REGISTER(32bits)
CODE	CS	IP
STACK	SS	SP
DATA	DS	SI or DI
EXTENDED DATA	ES	SI OR DI
EXTENDED DATA	FS	SI OR DI
EXTENDED DATA	GS	SI OR DI

MEMORY ADDRESSING - SEGMENTATION UNIT

❑ Questions

- * Describe the following :
 - i) Logical address ii) Linear address iii) Physical address
- * Explain how logical address is translated into physical address with the help of a block diagram.
- * What is MMU? Why is it required?
- * State the purpose of Memory Arbiter.
- * Describe the two phases in the translation of logical address to physical address.
- * Describe the purpose of the following :
 - a) Segmentation unit b) Paging unit
- * What are the two operating modes of intel microprocessor?
- * Explain the following with respect to x86 processor :
 - a) Real mode b) Protected mode.
- * A logical address consists of two parts and an that specifies the relative address within the segment
- * Explain what is meant by Segment Selector.
- * Six Segmentation registers provided in x86 processor to hold Segment Selectors are:
.....
- * Describe the Specific purpose of the three segment registers CS, DS and SS. Also indicate their corresponding offset registers.
- * Describe the three fields in the Segment Selector CS with the help of a figure.

SOC 3010

OPERATING SYSTEM

WEEK 3 LECTURE 2

MEMORY ADDRESSING

SEGMENT DESCRIPTORS

Memory Addressing

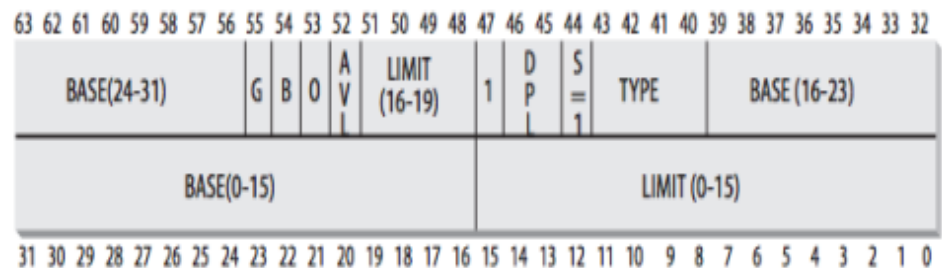
Segment Descriptors

Figure :Segment Descriptor Format

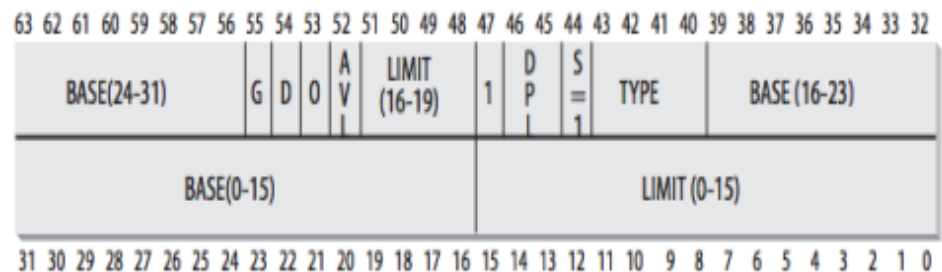
- Each segment is represented by an 8-byte Segment Descriptor that describes the segment characteristics.

- Segment Descriptors are stored either in the Global Descriptor Table (GDT) or in the Local Descriptor Table (LDT).

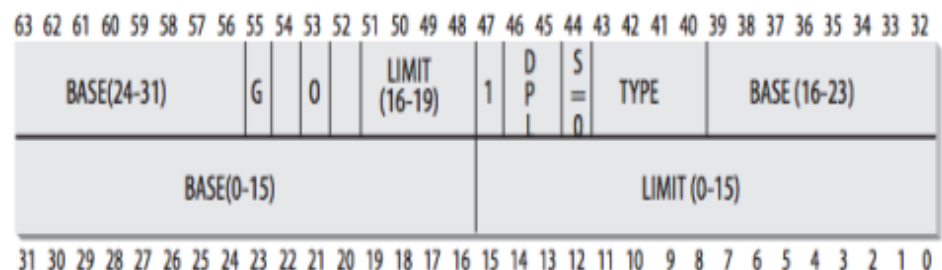
Data Segment Descriptor



Code Segment Descriptor



System Segment Descriptor



Memory Addressing

- * Usually **only one GDT** is defined, while each process may have its own LDT if it needs to create additional segments besides those stored in the GDT.
- * The address of the GDT in main memory is contained in the **gdtr** processor register and the address of the currently used LDT is contained in the **ldtr** processor register.
- * Each Segment Descriptor consists of the following fields:
 - A **32-bit Base** field that contains the linear address of the first byte of the segment.
 - A **20-bit Limit** field that denotes the segment length in bytes.
 - A **G granularity flag**: if it is cleared, the segment size is expressed in bytes; otherwise, it is expressed in multiples of 4096 bytes.
 - If G is set to 0, the size of a non-null segment may vary between 1 byte and 1 MB; otherwise, it may vary between 4 KB and 4 GB.
 - An **S system flag**: if it is cleared, the segment is a system segment that stores kernel data structures; otherwise, it is a normal code or data segment.
 - A **4-bit Type** field that characterizes the segment type and its access rights.

Memory Addressing

- A **DPL (Descriptor Privilege Level)** 2-bit field used to restrict accesses to the segment.

It represents the minimal CPU privilege level requested for accessing the segment.

Therefore, a segment with its DPL set to 0 is accessible only when the CPL is 0, that is, in Kernel Mode, while a segment with its DPL set to 3 is accessible with every CPL value.

- A **Segment-Present flag P** that is set to 0 if the segment is currently not stored in main memory.
- Linux always sets this field bit 47 to 1, since it never swaps out whole segments to disk.
- An **additional flag called D (Default) or B (Big)** depending on whether the segment contains code or data. Its meaning is slightly different in the two cases, **but it is basically set if the addresses used as segment offsets are 32 bits long and it is cleared if they are 16 bits long**
- A reserved bit (bit 53) always set to 0.
- An **AVL flag** that may be used by the operating system but is ignored in Linux.

Memory Addressing

TYPE: Distinguishes between various kinds of descriptors.

A - Accessed bit: The processor sets this bit when the segment is accessed; i.e., a selector for the descriptor is loaded into a segment register or used by a selector test instruction

W – Writable bit

-can Read from & Write to the segment

R - Readable bit

- Can only Read & Execute the code in the segment

E- Expand Down bit

If clear, the segment expands from base address up to base+limit. If set, it expands from maximum offset down to limit, a behavior usually used for stacks

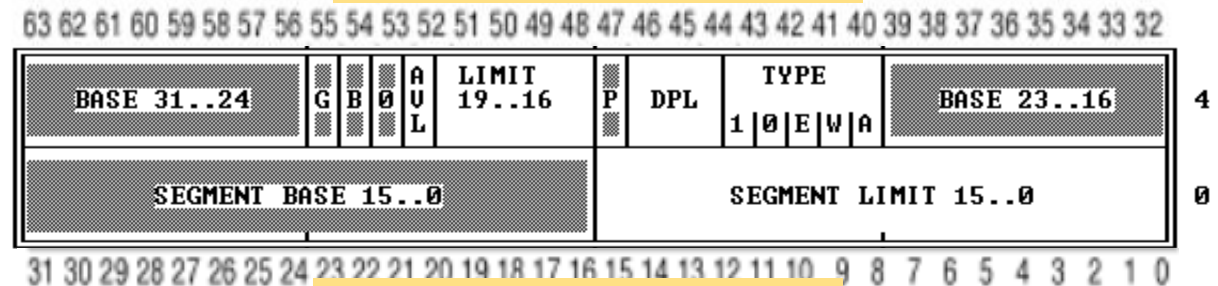
C – Conforming bit

Code in this segment may be called from less-privileged levels

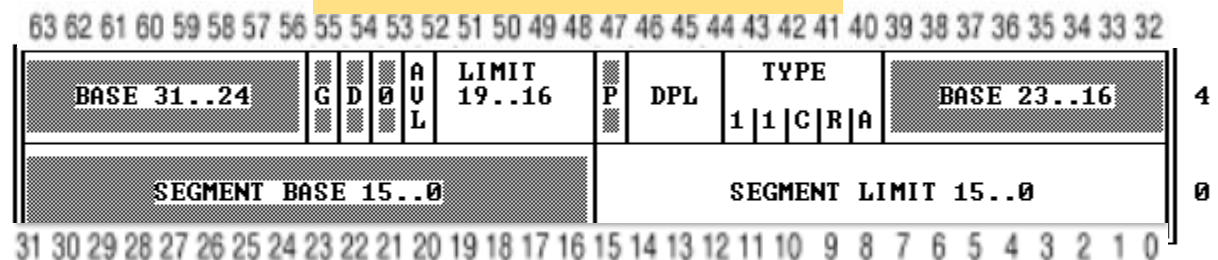
FIGURE

Protection Fields of Segment Descriptors

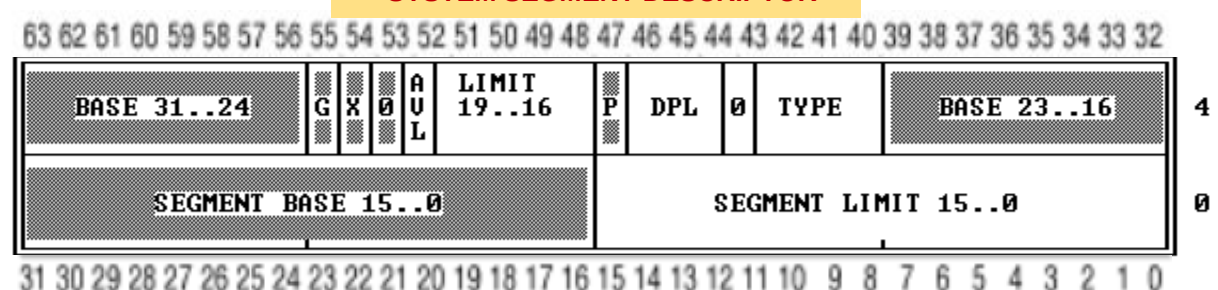
DATA SEGMENT DESCRIPTOR



CODE SEGMENT DESCRIPTOR



SYSTEM SEGMENT DESCRIPTOR



A – ACCESSED
 AVL – AVAILABLE FOR PROGRAMMERS USE
 B – BIG
 C – CONFORMING
 D – DEFAULT
 DPL – DESCRIPTOR PRIVILEGE LEVEL

E – EXPAND-DOWN
 G – GRANULARITY
 P – SEGMENT PRESENT
 R – READABLE
 W – WRITABLE

Memory Addressing

TYPE: In System Segment 4-bit Type field value 1011=(11)₁₀ indicates that current process is executing otherwise its value is 1001=9

DPL (Descriptor Privilege Level): Used by the protection mechanism

DPL – 00 means that this segment can be accessed in only Kernel Mode

DPL – 11 means that this segment can be accessed in both User & Kernel Mode

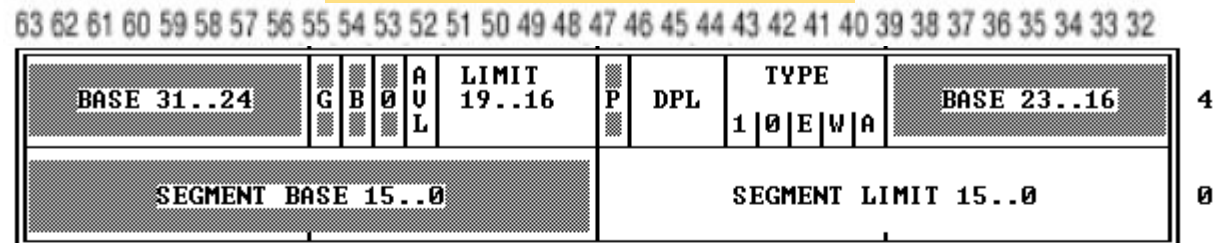
P-Segment-Present bit that is set to 0 if the segment is currently not stored in main memory.

- Linux always sets this field bit 47 to 1, since it never swaps out whole segments

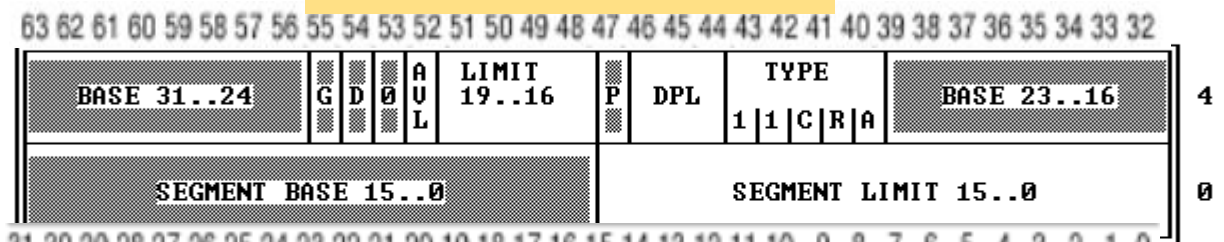
FIGURE

Protection Fields of Segment Descriptors

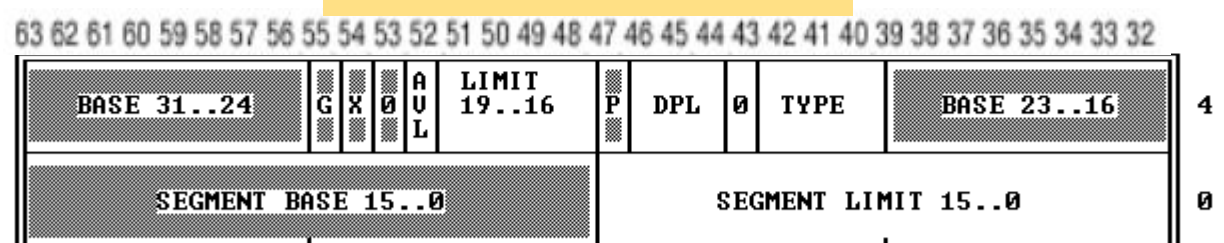
DATA SEGMENT DESCRIPTOR



CODE SEGMENT DESCRIPTOR



SYSTEM SEGMENT DESCRIPTOR



A – ACCESSED
AUL – AVAILABLE FOR PROGRAMMERS USE
B – BIG
C – CONFORMING
D – DEFAULT
DPL – DESCRIPTOR PRIVILEGE LEVEL

E – EXPAND-DOWN
G – GRANULARITY
P – SEGMENT PRESENT
R – READABLE
W – WRITABLE

Memory Addressing

- ✳ The following Segment Descriptor types are widely used:
 - ❑ **Code Segment Descriptor**
 - ✳ Indicates that the Segment Descriptor refers to a code segment; it may be included either in the GDT or in the LDT. The descriptor has the S flag set.
 - ❑ **Data Segment Descriptor**
 - ✳ Indicates that the Segment Descriptor refers to a data segment; it may be included either in the GDT or in the LDT. The descriptor has the S flag set.
 - ✳ Stack segments are implemented by means of generic data segments.

Memory Addressing

❑ Task State Segment Descriptor (TSSD)

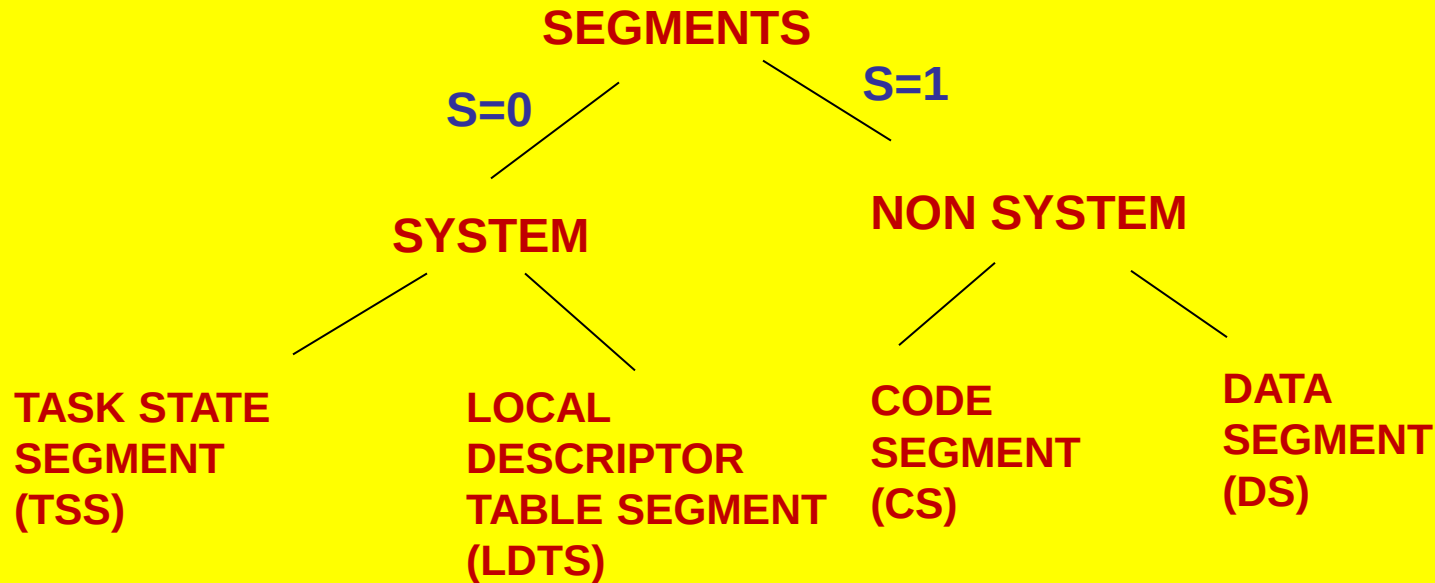
- ★ Indicates that the Segment Descriptor refers to a Task State Segment (TSS), that is, a segment used to save the contents of the processor registers; it can appear only in the GDT.
- ★ The corresponding Type field has the value 11 or 9, depending on whether the corresponding process is currently executing on the CPU.
- ★ The S flag of such descriptors is set to 0.

❑ Local Descriptor Table Descriptor (LDTD)

- ★ Indicates that the Segment Descriptor refers to a segment containing an LDT; it can appear only in the GDT. The corresponding Type field has the value 2. The S flag of such descriptors is set to 0.

Memory Addressing

Q)What are the two types of Segments?



Q) A segment descriptor entry in GDT contains the following fields :

Base (starting address of linear segment)= 0x20000000, Limit = 0xFFFF, G=0

a) Compute the size or length of the Linear Segment

Given Limit = 0xFFFF = 2^{20} , G=0, then size = limit bytes= 2^{20} bytes= 1MB

b) Determine the ending address of the Linear Segment

Size of the linear segment = 1MB = 2^{20} = 0xFFFF = limit

Ending address of the linear Segment = Starting address + limit

= 0x20000000 + 0xFFFF = 0x200FFFFF

Memory Addressing



Q) A segment descriptor entry in GDT contains the following fields :

Base (starting address of linear segment)= 0x00000000, Limit = 0xFFFFF, G=1

a) Compute the size or length of the Linear Segment

Given Limit = 0xFFFFF = 2^{20} , G=1, then size = limit pages = 2^{20} pages = $2^{20} \times 4\text{KB}$

Size or length of the linear segment = $2^{20} \times 2^{12} = 2^{32} = 4\text{GB}$

b) Determine the ending address of the Linear Segment

Size of the linear segment = 4GB = $2^{32} = 0xFFFFFFFF = \text{limit}$

Ending address of the linear Segment = Starting address + limit
= 0x00000000 + 0xFFFFFFFF = 0xFFFFFFFF

Q) A segment descriptor entry in GDT contains the following fields :

Base (starting address of linear segment)= 0x03000000, Limit = 0x3FFF, G=0

a) Compute the size or length of the Linear Segment

Given Limit = 0x3FFF = 2^{14} , G=0, then size = limit bytes = 2^{14} bytes = 16KB

Size or length of the linear segment = 16KB

b) Determine the ending address of the Linear Segment

Size of the linear segment = 16KB = $2^{14} = 0x3FFF = \text{limit}$

Ending address of the linear Segment = Starting address + limit
= 0x03000000 + 0x3FFF = 0x03003FFF

Q) A segment descriptor entry in GDT contains the following fields :

Base (starting address of linear segment)= 0x03000000, Limit = 0x3FFF, G=1

a) Compute the size or length of the Linear Segment

Given Limit = 0x3FFF = 2^{14} , G=1, then size = limit pages = 2^{14} pages = $2^{14} \times 4\text{KB} = 2^{14} \times 2^{12} = 2^{26}$

Size or length of the linear segment = $2^{26} = 64\text{MB}$

b) Determine the ending address of the Linear Segment

Size of the linear segment = 64MB = $2^{26} = 0x3FFFFFFF = \text{limit}$

Ending address of the linear Segment = Starting address + limit
= 0x03000000 + 0x3FFFFFFF = 0x06FFFFFFF

MEMORY ADDRESSING - SEGMENTATION UNIT

❑ Questions

- * State the purpose of the following fields in Segment Selector and also specify their size – INDEX, TI, RPL
- * The value 0 in 2-bit Current Privilege Level(CPL) field of CS register refers to...
- * The value 3 in 2-bit Current Privilege Level(CPL) field of CS register refers to ..
- * Explain what is meant by Segment Descriptor.
- * Draw the format of the 8-byte Code Segment Descriptor and explain the purpose of various fields in the descriptor.
- * Explain the purpose of following fields in a Segment Descriptor :
 - a)BASE b) LIMIT c) G d) DPL e) S f) P
 - g) TYPE h) D/B i) A j) E k)R/W
- * Explain what is meant by GDT. What does it contain?
- * What are System and Non-System Segments?
- * Describe the following Segment Descriptors. Clearly indicate whether they are System Segments or Non-Segment Segments. Also specify whether they appear only in GDT or only in LDT or both in GDT & LDT
 - a) Code Segment Descriptor
 - b) Data Segment Descriptor
 - c) Task State Segment Descriptor (TSSD)
 - d) Local Descriptor Table Descriptor (LDTD)

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 2 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG